Searching and Algorithms Analysis

Executing Time

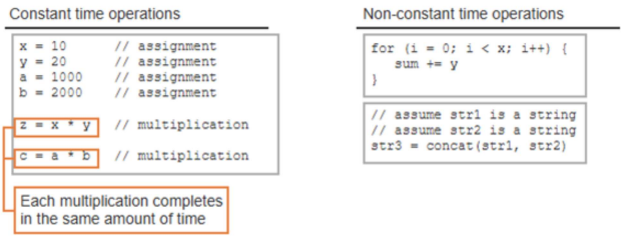
Suppose two algorithms perform the same task such as search (linear search vs. binary search). Which one is better? One possible approach to answer this question is to implement these algorithms in Java and run the programs to get execution time. But there are two problems for this approach:

- First, there are many tasks running concurrently on a computer. The execution time of a particular program is dependent on the system load.
- Second, the execution time is dependent on specific input. Consider linear search and binary search for example. If an element to be searched happens to be the first in the list, linear search will find the element quicker than binary search.

Growth Rate

- Analyze algorithms independent of computers and specific input.
- This approach approximates the effect of a change on the size of the input.
- How fast an algorithm's execution time increases as the input size increases.
- Compare two algorithms by examining their *growth rates*.

Constant Time Operations



4

Identifying constant time operations:

Arithmetic operations (+,-,*,/,%)

Assignment of a reference, pointer, or other fixed size data value.

Comparison of two fixed size data values.

Read or write an array element at a particular index.

Comparing Common Growth Functions

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$$

$O(1)$	Constant time
$O(\log n)$	Logarithmic time
$O(n)$	Linear time
$O(n \log n)$	Log-linear time
$O(n^2)$	Quadratic time
$O(n^3)$	Cubic time
$O(2^n)$	Exponential time

5

O(1) (if .. Else ..)

O(log n) binary search

O(n) linear search

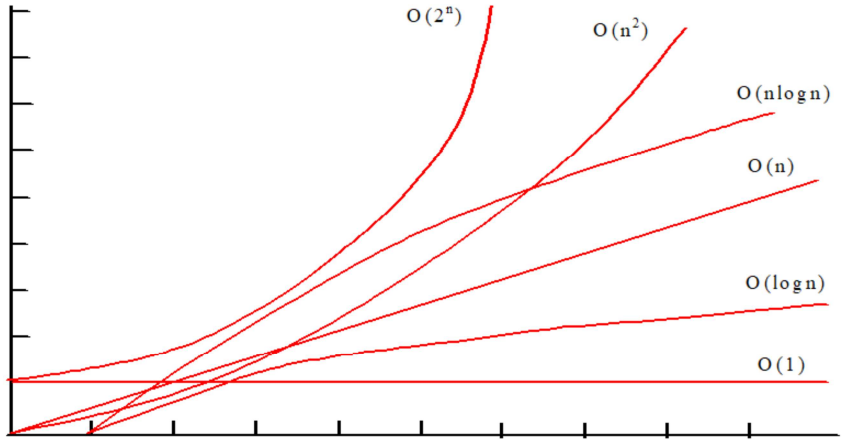
O(nlog n): mergeSort

O(n^2): Selection Sort

O(c^N): recursive Fibonacci , tower of Hanoi

Comparing Common Growth Functions

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$$



6

Ignoring Multiplicative Constants

The linear search algorithm requires *n* comparisons in the worst-case and *n/2* comparisons in the average-case. Using the Big *O* notation, both cases require *O(n)* time. The multiplicative constant (1/2) can be omitted. Algorithm analysis is focused on growth rate. The multiplicative constants have no impact on growth rates. The growth rate for n/2 or 100n is the same as n, i.e., O(n) = O(n/2) = O(100n).

f(*n*) / *n*	*n*	*n*/2	100*n*
100	100	50	10000
200	200	100	20000
	2	2	2

f(200) / *f*(100)

Ignoring Non-Dominating Terms

Consider the algorithm for finding the maximum number in an array of *n* elements. If *n* is 2, it takes one comparison to find the maximum number. If *n* is 3, it takes two comparisons to find the maximum number. In general, it takes *n-1* times of comparisons to find maximum number in a list of *n* elements. Algorithm analysis is for large input size. If the input size is small, there is no significance to estimate an algorithm's efficiency. As *n* grows larger, the *n* part in the expression *n-1* dominates the complexity. The Big *O* notation allows you to ignore the non-dominating part (e.g., -1 in the expression *n-1*) and highlight the important part (e.g., *n* in the expression *n-1*). So, the complexity of this algorithm is *O(n)*.

8

The seven useful functions

- The constant function
  - $f(n) = c$
- The logarithm function
  - $f(n) = \log_b n$, for some constant $b > 1$

  $x = \log_b n$ if and only if $b^x = n$
- Logarithm rules (given real numbers $a,c > 0$ and $b, d > 1$)
  1. $\log_b (ac) = \log_b a + \log_b c$
  2. $\log_b (a/c) = \log_b a - \log_b c$
  3. $\log_b (a^c) = c \log_b a$
  4. $\log_b a = \log_d a / \log_d b$
  5. $b^{\log_d a} = a^{\log_d b}$

9

As simple as it is, the constant function is useful in algorithm analysis because it characterizes the number of steps needed to do a basic operation on a computer, like adding two numbers, assigning a value to a variable, or comparing two numbers.

The value is known as the ***base*** of the logarithm. Note that by the above definition, for any base b > 1

$\log_b b = 1$

$\log_b 1 = 0$

Example 5.2.1

We demonstrate below some interesting applications of the logarithm rules from Proposition 5.2.1 (using the usual convention that the base of a logarithm is 2 if it is omitted).

- $\log(2n) = \log 2 + \log n = 1 + \log n$, by rule 1
- $\log(n/2) = \log n - \log 2 = (\log n) - 1$, by rule 2
- $\log n^3 = 3 \log n$, by rule 3
- $\log 2^n = n \log 2 = n \cdot 1 = n$, by rule 3
- $\log_4 n = (\log n)/\log 4 = (\log n)/2$, by rule 4
- $2^{\log n} = n^{\log 2} = n^1 = n$, by rule 5

Feedback?

Useful Mathematic Summations

$$1+2+3+....+(n-1)+n=\frac{n(n+1)}{2}$$

$$a^0+a^1+a^2+a^3+....+a^{(n-1)}+a^n=\frac{a^{n+1}-1}{a-1}$$

$$2^0+2^1+2^2+2^3+....+2^{(n-1)}+2^n=\frac{2^{n+1}-1}{2-1}$$

Check Point

- What is the order of each of the following:
  - $\frac{(n^2+1)^2}{n}$
  - $\frac{n^2+(\log 2)n^4}{n}$
  - $n^3 + 100n^2 + n$
  - $2^n + 100n^2$+45n
  - n + $2^n + 2^n\ 2^n$

Determining Big-O
worst-case algorithm analysis

- Repetition
- Sequence
- Selection
- Logarithm

Repetition: Simple Loops

executed
n times

```
for (i = 1; i <= n; i++) {
  k = k + 5;
}
```

constant time

Time Complexity

T(n) = (a constant c) * n = cn = **O(n)**

Ignore multiplicative constants (e.g., "c").

Repetition: Nested Loops

executed *n* times

```
for (i = 1; i <= n; i++) {
  for (j = 1; j <= n; j++) {
    k = k + i + j;
  }
}
```

inner loop executed *n* times

constant time

Time Complexity

T(n) = (a constant c) * n * n = cn^2 = $O(n^2)$

Ignore multiplicative constants (e.g., "c").

15

Repetition: Nested Loops

executed *n* times

```
for (i = 1; i <= n; i++) {
  for (j = 1; j <= 20; j++) {
    k = k + i + j;
  }
}
```

inner loop executed *20* times

constant time

Time Complexity

$T(n) = 20 * c * n = O(n)$

*Ignore multiplicative constants (e.g., 20*c)*

Check Point

```
for (int i = 0; i < list.length - 1; i++) {
      // Find the minimum in the list[i..list.length-1]
      double currentMin = list[i];
      int currentMinIndex = i;

      for (int j = i + 1; j < list.length; j++) {
        if (currentMin > list[j]) {
          currentMin = list[j];
          currentMinIndex = j;
        }
      }

      // Swap list[i] with list[currentMinIndex] if necessary;
      if (currentMinIndex != i) {
        list[currentMinIndex] = list[i];
        list[i] = currentMin;
      }
    }
```

Sequence

executed *10* times

```
for (j = 1; j <= 10; j++) {
  k = k + 4;
}
```

executed *n* times

```
for (i = 1; i <= n; i++) {
  for (j = 1; j <= 20; j++) {
    k = k + i + j;
  }
}
```

inner loop executed *20* times

Time Complexity

T(n) = c *10 + 20 * c * n = O(n)

Selection

O(n)

```
if (list.contains(e)) {
  System.out.println(e);
}
else
  for (Object t: list) {
    System.out.println(t);
  }
```

Let n be list.size(). Executed n times.

Time Complexity

T(n) = test time + worst-case (if, else)
= O(n) + O(n)
= O(n)

Logarithm: Analyzing Binary Search

$n = 2^k$

The binary search algorithm presented in Lecture, BinarySearch.java, searches a key in a sorted array. Each iteration in the algorithm contains a fixed number of operations, denoted by *c*. Let *T(n)* denote the time complexity for a binary search on a list of *n* elements. Without loss of generality, assume *n* is a power of 2 and *k=log n*. Since binary search eliminates half of the input after two comparisons,

$$T(n) = T(\frac{n}{2}) + c = T(\frac{n}{2^2}) + c + c = \ldots = T(\frac{n}{2^k}) + ck = T(1) + c\log n = 1 + c\log n$$

$$= O(\log n)$$

20

Logarithmic Time

Ignoring constants and smaller terms, the complexity of the binary search algorithm is *O(logn)*. An algorithm with the *O(logn)* time complexity is called a *logarithmic algorithm*. The base of the log is 2, but the base does not affect a logarithmic growth rate, so it can be omitted. The logarithmic algorithm grows slowly as the problem size increases. If you square the input size, you only double the time for the algorithm.

Check Point

- Count the number of iterations in the following loops:

a)
```
int cnt = 1;
while ( cnt < 30 )
  cnt = cnt * 2;
```

b)
```
int cnt = 15;
while ( cnt < 30 )
  cnt = cnt * 3;
```

c)
```
int cnt = 1;
while ( cnt < n )
  cnt = cnt * 2;
```

b)
```
int cnt = 15;
while ( cnt < n )
  cnt = cnt * 3;
```

Check Point

- Use the Big O notation to estimate the time complexity of the following methods:
  - public static void m(int n)
    {
    for(int i=0; i<n; ++i){
    System.out.println(Math.random());
    }
    }
  - public static void m(int n)
    {
    for(int i=0; i<n; ++i){
    for(int j=0; j<i; ++j)
    System.out.println(Math.random());
    }
    }

Check Point

```
(c) public static void mC(int[] m) {
     for (int i = 0; i < m.length; i++) {
      System.out.print(m[i]);
     }
     for (int i = m.length - 1; i >= 0; ) {
       System.out.print(m[i]); i--; }
     }

(d) public static void mD(int[] m) {
      for (int i = 0; i < m.length; i++) {
        for (int j = 0; j < i; j++)
          System.out.print(m[i] * m[j]);
      }
    }
```

Check Point

- The number of operations executed by algorithms A and B is $8n \log n$ and $2n^2$, respectively. Determine n_0 such that A is better than B for $n >= n_0$

24

8nlogn = 2n^2

n=4logn

N=2^4=16

Check Point

For each of the following code fragments, identify the running time in terms of variable n

```
int total=0;
for (int j=0; j < n; j++) {
  total += data[j];
}
int big=data[0];
for (int k=1; k < n; k++)
{
  big = Math.max(big, data[k]);
}
```

Check Point

For each of the following code fragments, identify the running time in terms of variable n

```
int gaps=0;
for (int j=0; j < n; j++)
{
   for (int k=j; k < n; k++) {
   // note k=j init
   gaps += k-j;
   }
}
```

Check Point

For each of the following code fragments, identify the running time in terms of variable n

```
int steps=0;
for (int k=1; k < n; k *= 2)
{ // note k*=2 update
  for (int j=0; j < k; j++) {
  // note j<k condition
  steps++;
  }
}
```

Analyzing Tower of Hanoi

The Tower of Hanoi problem presented in Listing 18.7, TowerOfHanoi.java, moves *n* disks from tower A to tower B with the assistance of tower C recursively as follows:

- Move the first *n – 1* disks from A to C with the assistance of tower B.
- Move disk *n* from A to B.
- Move *n - 1* disks from C to B with the assistance of tower A.

Let *T(n)* denote the complexity for the algorithm that moves disks and c denote the constant time to move one disk, i.e., T(1) is c. So,

$$T(n) = T(n-1) + c + T(n-1) = 2T(n-1) + c$$
$$= 2(2(T(n-2) + c) + c) = 2^{n-1}T(1) + c2^{n-2} + \ldots + c2 + c =$$
$$= c2^{n-1} + c2^{n-2} + \ldots + c2 + c = c(2^n - 1) = O(2^n)$$

28

Case Study: Fibonacci Numbers

```
/** The method for finding the Fibonacci number */
public static long fib(long index) {
  if (index == 0) // Base case
    return 0;
  else if (index == 1) // Base case
    return 1;
  else  // Reduction and recursive calls
    return fib(index - 1) + fib(index - 2);
}
```

Case Study: Non-recursive version of Fibonacci Numbers

```
public static long fib(long n) {
  long f0 = 0; // For fib(0)
  long f1 = 1; // For fib(1)
  long f2 = 1; // For fib(2)

  if (n == 0)
    return f0;
  else if (n == 1)
    return f1;
  else if (n == 2)
    return f2;

  for (int i = 3; i <= n; i++) {
    f0 = f1;
    f1 = f2;
    f2 = f0 + f1;
  }

  return f2;
}
```

30

Obviously, the complexity of this new algorithm is O(n) . This is a tremendous improvement over the recursive algorithm.

Case Study: GCD Algorithms Version 1

```
public static int gcd(int m, int n) {
  int gcd = 1;
  for (int k = 2; k <= m && k <= n; k++) {
    if (m % k == 0 && n % k == 0)
      gcd = k;
  }
  return gcd;
}
```

$O(n)$

Case Study: GCD Algorithms Version 2

```
for (int k = n; k >= 1; k--) {
  if (m % k == 0 && n % k == 0) {
    gcd = k;
    break;
  }
}
```

Euclid's Algorithm Implementation

```
public static int gcd(int m, int n) {
  if (m % n == 0)
    return n;
  else
    return gcd(n, m % n);
}
```

33